

dsu on tree

梁宝烨

佛山市南海区石门中学

2025 年 7 月 15 日

目录

1 引入

2 算法实现

3 练习

dsu on tree 是一种处理树上不带修改，离线的子树问题的算法，时间复杂度通常是 $O(n \log n)$ 。

dsu on tree 是一种处理树上不带修改，离线的子树问题的算法，
时间复杂度通常是 $O(n \log n)$ 。
中文名是树上启发式合并。

启发式合并

- 启发式算法是什么？

启发式合并

- 启发式算法是什么？
- 启发式算法是基于人类的经验和直观感觉，对一些算法的优化。

启发式合并

- 启发式算法是什么？
- 启发式算法是基于人类的经验和直观感觉，对一些算法的优化。
- 例如启发式合并：要合并两个集合时，暴力把小的集合中的元素一个一个加进大的里面。

启发式合并

- 启发式算法是什么？
- 启发式算法是基于人类的经验和直观感觉，对一些算法的优化。
- 例如启发式合并：要合并两个集合时，暴力把小的集合中的元素一个一个加进大的里面。
- 如果有若干个集合要合并，且所有集合的大小加起来不超过 n ，暴力加入的次数是 $O(n \log n)$ 的。

启发式合并

- 启发式算法是什么？
- 启发式算法是基于人类的经验和直观感觉，对一些算法的优化。
- 例如启发式合并：要合并两个集合时，暴力把小的集合中的元素一个一个加进大的里面。
- 如果有若干个集合要合并，且所有集合的大小加起来不超过 n ，暴力加入的次数是 $O(n \log n)$ 的。
- 因为当一个小集合被暴力加进大的里面后，相当于大小翻倍了。

树上？

回到 dsu on tree, 在树上要启发式合并啥？

树上？

回到 dsu on tree, 在树上要启发式合并啥？
我们可以这样子，对于一个结点，我们把它所有小儿子往大儿子上“合并”。

树上？

回到 dsu on tree，在树上要启发式合并啥？

我们可以这样子，对于一个结点，我们把它所有小儿子往大儿子上“合并”。

何以区分小大？还记得第一页讲的吗，dsu on tree 是用来解决子树问题的。

树上？

回到 dsu on tree，在树上要启发式合并啥？

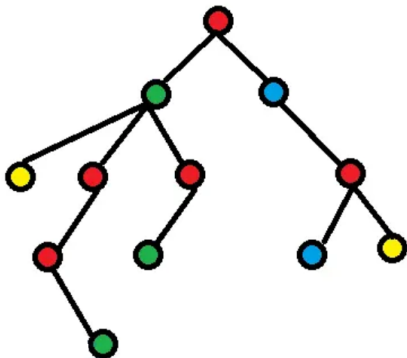
我们可以这样子，对于一个结点，我们把它所有小儿子往大儿子上“合并”。

何以区分小大？还记得第一页讲的吗，dsu on tree 是用来解决子树问题的。

所以用子树大小作为评判标准，就和树剖一样，大儿子就是重儿子，小儿子就是轻儿子。

从一道例题讲起

考虑下面的问题：给出一棵树，每个节点有颜色，询问一些子树的颜色数量（颜色可重复）。



暴力

考虑 $O(n^2)$ 的做法，直接暴力计算每个点子树内的颜色数。

暴力

考虑 $O(n^2)$ 的做法，直接暴力计算每个点子树内的颜色数。
做法就是开一个 bool 的 vis 数组记录每种颜色是否出现过，然后
对于每个点 u ，暴力遍历子树计算。
算完后清空 vis 数组，然后递归往下计算 u 的儿子的答案。

暴力做法慢就慢在进行了很多重复遍历，没有高效利用信息。

暴力做法慢就慢在进行了很多重复遍历，没有高效利用信息。
那我们能不能先递归算儿子，不清空 vis 数组，最后再加上 u 自身来计算 u 的答案？

暴力做法慢就慢在进行了很多重复遍历，没有高效利用信息。
那我们能不能先递归算儿子，不清空 vis 数组，最后再加上 u 自身来计算 u 的答案？
显然并不行，这个 vis 数组是全局的，不清空则会记录来自子树外的颜色信息。

忽略空间问题

那么对于每个子树都开一个单独的 `vis` 数组呢？然后 `u` 的信息可以由它的儿子 `v` 们继承过来。

忽略空间问题

那么对于每个子树都开一个单独的 `vis` 数组呢？然后 `u` 的信息可以由它的儿子 `v` 们继承过来。

如果不考虑空间的问题，事实上这是可行的，我们可以把 `u` 的重儿子的 `vis` 数组拿来当做 `u` 自己的（反正重儿子的答案已经计算完毕），这部分是 $O(1)$ 的复杂度。

忽略空间问题

那么对于每个子树都开一个单独的 `vis` 数组呢？然后 `u` 的信息可以由它的儿子 `v` 们继承过来。

如果不考虑空间的问题，事实上这是可行的，我们可以把 `u` 的重儿子的 `vis` 数组拿来当做 `u` 自己的（反正重儿子的答案已经计算完毕），这部分是 $O(1)$ 的复杂度。

然后对于轻儿子的 `vis` 数组就暴力合并上来。

忽略空间问题

那么对于每个子树都开一个单独的 `vis` 数组呢？然后 `u` 的信息可以由它的儿子 `v` 们继承过来。

如果不考虑空间的问题，事实上这是可行的，我们可以把 `u` 的重儿子的 `vis` 数组拿来当做 `u` 自己的（反正重儿子的答案已经计算完毕），这部分是 $O(1)$ 的复杂度。

然后对于轻儿子的 `vis` 数组就暴力合并上来。

这样子暴力合并的复杂度是 $O(n \log n)$ 的，为什么？

忽略空间问题

那么对于每个子树都开一个单独的 `vis` 数组呢？然后 `u` 的信息可以由它的儿子 `v` 们继承过来。

如果不考虑空间的问题，事实上这是可行的，我们可以把 `u` 的重儿子的 `vis` 数组拿来当做 `u` 自己的（反正重儿子的答案已经计算完毕），这部分是 $O(1)$ 的复杂度。

然后对于轻儿子的 `vis` 数组就暴力合并上来。

这样子暴力合并的复杂度是 $O(n \log n)$ 的，为什么？

因为每个点到根上最多只有 $O(\log n)$ 条轻边，且每经过一个轻边会被暴力遍历一次（轻儿子要暴力合并）。

忽略空间问题

那么对于每个子树都开一个单独的 `vis` 数组呢？然后 `u` 的信息可以由它的儿子 `v` 们继承过来。

如果不考虑空间的问题，事实上这是可行的，我们可以把 `u` 的重儿子的 `vis` 数组拿来当做 `u` 自己的（反正重儿子的答案已经计算完毕），这部分是 $O(1)$ 的复杂度。

然后对于轻儿子的 `vis` 数组就暴力合并上来。

这样子暴力合并的复杂度是 $O(n \log n)$ 的，为什么？

因为每个点到根上最多只有 $O(\log n)$ 条轻边，且每经过一个轻边会被暴力遍历一次（轻儿子要暴力合并）。

事实上这就是 `dsu on tree` 的基本做法了。

完整的 dsu on tree

理想很美好，现实很残酷，我们不能忽略空间问题。

完整的 dsu on tree

理想很美好，现实很残酷，我们不能忽略空间问题。
既然不可能每个点都开个 vis 数组，就只能开一个全局的，我们需要想办法高效利用空间。

完整的 dsu on tree

理想很美好，现实很残酷，我们不能忽略空间问题。
既然不可能每个点都开个 vis 数组，就只能开一个全局的，我们需要想办法高效利用空间。
下面给出 dsu on tree 的解决方案：

完整的 dsu on tree

理想很美好，现实很残酷，我们不能忽略空间问题。
既然不可能每个点都开个 vis 数组，就只能开一个全局的，我们需要想办法高效利用空间。

下面给出 dsu on tree 的解决方案：

- 首先，信息和答案是被保存在全局的。

完整的 dsu on tree

理想很美好，现实很残酷，我们不能忽略空间问题。
既然不可能每个点都开个 vis 数组，就只能开一个全局的，我们需要想办法高效利用空间。

下面给出 dsu on tree 的解决方案：

- 首先，信息和答案是被保存在全局的。
- 对于结点 u ，先递归处理好它的所有轻儿子及轻儿子的子树的答案。

完整的 dsu on tree

理想很美好，现实很残酷，我们不能忽略空间问题。
既然不可能每个点都开个 vis 数组，就只能开一个全局的，我们需要想办法高效利用空间。

下面给出 dsu on tree 的解决方案：

- 首先，信息和答案是被保存在全局的。
- 对于结点 u ，先递归处理好它的所有轻儿子及轻儿子的子树的答案。
- 算好一个轻儿子的答案立马删除它的所有信息和贡献。

完整的 dsu on tree

理想很美好，现实很残酷，我们不能忽略空间问题。
既然不可能每个点都开个 vis 数组，就只能开一个全局的，我们需要想办法高效利用空间。

下面给出 dsu on tree 的解决方案：

- 首先，信息和答案是被保存在全局的。
- 对于结点 u ，先递归处理好它的所有轻儿子及轻儿子的子树的答案。
- 算好一个轻儿子的答案立马删除它的所有信息和贡献。
- 然后再算重儿子和重儿子的子树的答案，算好后不删除它的信息和贡献。

完整的 dsu on tree

理想很美好，现实很残酷，我们不能忽略空间问题。
既然不可能每个点都开个 vis 数组，就只能开一个全局的，我们需要想办法高效利用空间。

下面给出 dsu on tree 的解决方案：

- 首先，信息和答案是被保存在全局的。
- 对于结点 u ，先递归处理好它的所有轻儿子及轻儿子的子树的答案。
- 算好一个轻儿子的答案立马删除它的所有信息和贡献。
- 然后再算重儿子和重儿子的子树的答案，算好后不删除它的信息和贡献。
- 最后暴力统计轻儿子的信息和贡献。

完整的 dsu on tree

理想很美好，现实很残酷，我们不能忽略空间问题。
既然不可能每个点都开个 vis 数组，就只能开一个全局的，我们需要想办法高效利用空间。

下面给出 dsu on tree 的解决方案：

- 首先，信息和答案是被保存在全局的。
- 对于结点 u ，先递归处理好它的所有轻儿子及轻儿子的子树的答案。
- 算好一个轻儿子的答案立马删除它的所有信息和贡献。
- 然后再算重儿子和重儿子的子树的答案，算好后不删除它的信息和贡献。
- 最后暴力统计轻儿子的信息和贡献。
- 时间复杂度 $O(n \log n)$ 。

伪代码

```
1 // u为当前结点，f为u的父亲，keep表示是否要删除信息（false表示删除）
2 void dfs(int u, int f, bool keep) {
3     for (v is u.lightson)
4         dfs(v, u, false);
5     if (u have heavy son) dfs(u.heavyson, u, true);
6     count(); // 统计u的轻儿子的信息，并计算u的答案
7     if (keep == false) del(); // 如果keep为false，删除u的信息
8 }
```

模板题：CF600E Lomsat gelral

见代码

CF741D

CF741D

重排后回文等价于最多只有一个字母的出现次数为奇数。

CF741D

重排后回文等价于最多只有一个字母的出现次数为奇数。
而题目中有 22 种字母，所以有 23 种情况。

CF741D

重排后回文等价于最多只有一个字母的出现次数为奇数。
而题目中有 22 种字母，所以有 23 种情况。
记 $S[u]$ 表示 u 到 1 的路径上的每种字符的奇偶状态。

CF741D

重排后回文等价于最多只有一个字母的出现次数为奇数。

而题目中有 22 种字母，所以有 23 种情况。

记 $S[u]$ 表示 u 到 1 的路径上的每种字符的奇偶状态。

对于一条路径 (x, y) , $S[x] \oplus S[y]$ 就是路径的信息，而长度则是 $\text{dep}[x] + \text{dep}[y] - 2 * \text{dep}[\text{lca}]$ 。

CF741D

重排后回文等价于最多只有一个字母的出现次数为奇数。

而题目中有 22 种字母，所以有 23 种情况。

记 $S[u]$ 表示 u 到 1 的路径上的每种字符的奇偶状态。

对于一条路径 (x, y) , $S[x] \oplus S[y]$ 就是路径的信息，而长度则是 $\text{dep}[x] + \text{dep}[y] - 2 * \text{dep}[\text{lca}]$ 。

并且我们钦定 (x, y) 会在 lca 处被计算到。

CF741D

重排后回文等价于最多只有一个字母的出现次数为奇数。

而题目中有 22 种字母，所以有 23 种情况。

记 $S[u]$ 表示 u 到 1 的路径上的每种字符的奇偶状态。

对于一条路径 (x, y) , $S[x] \oplus S[y]$ 就是路径的信息，而长度则是 $\text{dep}[x] + \text{dep}[y] - 2 * \text{dep}[\text{lca}]$ 。

并且我们钦定 (x, y) 会在 lca 处被计算到。

所以我们在 dsu on tree 的时候开个桶存入 dep 即可。

CF741D

重排后回文等价于最多只有一个字母的出现次数为奇数。

而题目中有 22 种字母，所以有 23 种情况。

记 $S[u]$ 表示 u 到 1 的路径上的每种字符的奇偶状态。

对于一条路径 (x, y) , $S[x] \oplus S[y]$ 就是路径的信息，而长度则是 $\text{dep}[x] + \text{dep}[y] - 2 * \text{dep}[\text{lca}]$ 。

并且我们钦定 (x, y) 会在 lca 处被计算到。

所以我们在 dsu on tree 的时候开个桶存入 dep 即可。

类似题：CF570D Tree Requests

双倍经验，都是要查询子树内某个深度的点的数量。

CF1009F Dominant Indices

CF208E Blood Cousins

以及维护信息难一点的题：P3605 [USACO17JAN] Promotion
Counting P

总结：

其实 dsu on tree 本质上就是个暴力，不同的题主要考察的是如何存储信息，计算贡献。

在更难的 dsu on tree 题中往往会使用平衡树等更复杂的数据结构去维护信息。

基本上只要满足：离线，子树查询，信息可以方便的插入删除等，很大概率就可以使用 dsu on tree 算法。